

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 05 -

NAMING CONVENTIONS E ESTILO DE CÓDIGO (PEP8)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	22
O QUE VOCÊ VIU NESTA AULA?	26
REFERÊNCIAS.....	28

LEANDRO

O QUE VEM POR AÍ?

Imagine um quarto cheio de brinquedos onde apenas você sabe onde cada item está guardado. Para outra pessoa, achar algo seria demorado e frustrante. Mas se tudo estiver organizado em prateleiras com etiquetas claras (carrinhos na prateleira dos carrinhos, lápis na gaveta de lápis, jogos de tabuleiro no armário de jogos), qualquer um encontra o que precisa rapidamente.

No código-fonte, ocorre o mesmo: sem um padrão de nomenclatura e estilo, só o autor entende plenamente onde está cada “peça” do programa. Entrar em um projeto sem padrões é como tentar achar um brinquedo em um quarto bagunçado – inevitavelmente caótico.

Felizmente, a comunidade Python estabeleceu um “guia de estilo” justamente para evitar essa bagunça. O PEP 8 – Python Enhancement Proposal nº 8 – define convenções de nomenclatura e formatação que tornam o código mais limpo, legível e consistente. Ao segui-las, todos os desenvolvedores e desenvolvedoras falam o mesmo idioma no código, facilitando a colaboração e a manutenção. Como diz o Zen of Python: “Readability counts” (“Legibilidade importa”).

De fato, código é lido com muito mais frequência do que escrito – por você mesmo, dias ou semanas depois, e por colegas que nunca o viram antes. Nesta aula, vamos refletir sobre os problemas que as convenções vêm resolver e conhecer técnicas para escrever um código Python impecável. Prepare-se e continue com a gente, pois teremos um hands on prático e materiais em vídeo para consolidar tudo isso!

HANDS ON

Vamos colocar a mão na massa! Nas videoaulas, vamos analisar um pequeno script Python repleto de violações do PEP 8 e então corrigi-lo passo a passo, aplicando as convenções de estilo. A ideia é mostrar, na prática, como identificar problemas de formatação e nomenclatura e, em seguida, usar ferramentas automáticas para saná-los.

Considere este código escrito sem seguir as convenções de estilo do Python:

```
def SomaValores(a,b):  
    result=a+b  
    return(result)  
x= 10; y=20  
z=SomaValores(x,y)
```

Código-fonte 1 – Exemplo de código Python com problemas de estilo (não aderente ao PEP8)
Fonte: Elaborado pelo autor (2025)

Mesmo sendo funcional, esse código apresenta vários problemas de legibilidade e estilo. Podemos notar: nome de função em maiúsculas (SomaValores), ausência de espaços após vírgulas e em torno do operador =, uso de ponto e vírgula para múltiplas instruções na mesma linha, indentação inconsistente (2 espaços em vez de 4) e falta de linhas em branco separando partes do código. Ao rodar um verificador de estilo, como o flake8, surgiriam diversos avisos apontando essas infrações – por exemplo, reclamações de “missing whitespace after ','” e “expected 2 blank lines, found 0”, entre outros. Também poderíamos usar o pylint, que acusaria o nome de função fora do padrão (pois SomaValores deveria ser em minúsculas). Esses feedbacks automáticos nos guiam nas correções necessárias.

Em seguida, aplicamos as correções recomendadas. Renomeamos a função e as variáveis para seguir o padrão snake_case, ajustamos a indentação para 4 espaços, inserimos quebras de linha apropriadas e removemos caracteres desnecessários. Uma ferramenta de formatação automática como o black pode ajudar a refatorar o código, especialmente na parte de espaçamento e quebras de linha. Após as modificações, nosso script fica assim:

```
def soma_valores(a: float, b: float) -> float:  
    """Calcula a soma de dois valores."""  
    resultado = a + b
```

```
    return resultado

x = 10
y = 20
z = soma_valores(x, y)
```

Código-fonte 2 – O mesmo código refatorado seguindo as convenções do PEP8 (nomes em snake_case, indentação, espaços, docstring etc.)

Fonte: Elaborado pelo autor (2025)

Compare as duas versões. A segunda está bem mais limpa e compreensível: os nomes das funções e variáveis indicam claramente seu propósito, a formatação destaca a estrutura lógica (indentação correta, linhas separando blocos) e até adicionamos uma breve docstring explicativa.

Nos vídeos, você poderá acompanhar detalhadamente cada etapa dessa transformação – desde a detecção dos problemas até o uso do flake8 e do black. Depois, tente reproduzir o processo em um código seu. Essa prática guiada vai ajudá-lo(a) a firmar os conhecimentos de convenções de estilo e ferramentas automáticas em Python.

SAIBA MAIS

Vamos nos aprofundar nos conceitos e racionalidades por trás do uso de convenções de nomenclatura e estilo de código em Python, bem como explorar ferramentas que nos ajudam a manter essas boas práticas de forma consistente. Seguir um padrão como o PEP 8 não é mero pedantismo – há fundamentos sólidos de engenharia de software, usabilidade e até alguns toques de teoria da computação por trás de cada recomendação. A seguir, discutiremos cada tópico com detalhe técnico e exemplos ilustrativos, relacionando sempre com situações reais de projetos (inclusive de ciência de dados) e citando fontes confiáveis para embasar as ideias.

Convencões de Nomenclatura (Naming Conventions) – um dos pilares do PEP 8 é padronizar como batizamos variáveis, funções, classes etc. Escolher bons nomes em código é uma arte e uma ciência: nomes descritivos e consistentes servem como documentação viva do programa. Por outro lado, nomes ruins podem introduzir ambiguidades terríveis (por exemplo, usar `l` como nome de variável pode ser confundido com o número 1 ou a letra `l` dependendo da fonte utilizada). O PEP 8 define convenções claras para minimizar esses problemas. Em resumo:

- **Variáveis e funções** devem ser nomeadas em letras minúsculas, com palavras separadas por sublinhado `_` (o estilo chamado `snake_case`). Exemplos: `valor_total`, `calcula_media()`. Nomes assim descrevem propósito (como `calcula_media` sugere uma função de cálculo de média) e são fáceis de ler. Evite usar abreviações obscuras ou nomes de uma letra só (a não ser variáveis muito temporárias, como índices em loops). “Explícito é melhor que implícito”, diz o Zen of Python; logo, prefira `quantidade_clientes` em vez de `qc`. Uma dica: nunca use `l`, `O` ou `I` isoladamente – como mencionado, eles se confundem visualmente com 1 e 0.
- **Classes** devem usar **CamelCase** (também chamado `PascalCase`), ou seja, cada palavra começa em maiúscula e não se usa sublinhado. Exemplo: `ClienteVIP` ou `DataLoader`. Isso distingue facilmente classes de funções/variáveis no código.
- **Constantes** (valores que não devem mudar) são definidas com **todas as letras maiúsculas**, separando palavras com `_`. Ex: `TAXA_DESCONTO`,

MAX_ITENS. Embora o Python não tenha constantes imutáveis de verdade, essa convenção sinaliza ao leitor que aquele valor deve ser tratado como constante.

- **Módulos** (arquivos .py) e **pacotes** (diretórios) também têm convenções: por serem objetos de alto nível, recomenda-se nomes curtos, em minúsculas. Para módulos, pode-se usar underscore se necessário (ex.: meu_modulo.py), já pacotes geralmente não utilizam underscores (ex.: diretório analisados/). Esses padrões evitam conflitos e facilitam o import.
- **Nomes “privados”**: em Python, usa-se um underscore inicial para indicar que uma variável ou método é interno (não parte da API pública). Por exemplo, `_segredo` ou `_calcula_dado()` sugere uso apenas dentro do módulo/classe. Dois underscores iniciais ativam name mangling em classes (`__atributo` vira um nome ofuscado como `_Classe__atributo`), recurso usado raramente, apenas para evitar colisões em herança. O PEP 8 menciona essas convenções para ajudar a comunicar intenções de encapsulamento.

A tabela a seguir resume essas convenções de nomenclatura com exemplos:

ELEMENTO	CONVENÇÃO (PEP8)	EXEMPLOS
Variável	minúsculas, usar _ entre palavras	saldo, total_vendas
Função	minúsculas, usar _ (snake_case)	calcula_media(), processar_dados()
Classe	CamelCase (Capitalize cada palavra)	Cliente, AnaliseDados
Constante	MAIÚSCULAS, usar _ entre palavras	TAXA_DESCONTO, MAX_ITENS

Módulo (arquivo)	minúsculas (usar _ se necessário)	utils.py, processamento_dados.py
Pacote (pasta)	minúsculas (evitar underscores)	relatorios (pacote)

Tabela 1 – Convenções de nomenclatura recomendadas pelo PEP 8 em Python

Fonte: Elaborado pelo autor (2025), com base em PEP8 e referências

Essas convenções trazem coerência semântica ao código. Pense nas classes e funções como capítulos e seções de um livro técnico: seguir o mesmo estilo de títulos e subtítulos (nomeação) ajuda o leitor a navegar e entender a estrutura. Em um projeto de ciência de dados, por exemplo, você pode ter classes como `DataPrepper` e `ModelTrainer` e funções como `carregar_dados()` e `avaliar_modelo()`. Seguindo o PEP 8, qualquer cientista ou engenheiro(a) que ler seu código saberá, só pelos nomes, o que são entidades (classes) e o que são ações ou cálculos (funções) e qual informação é constante. Isso reduz a carga cognitiva e evita erros – dificilmente alguém vai confundir uma função com uma variável, ou usar um objeto errado, se os nomes estão claros.

Indentação e Formatação Geral: o Python é notório por usar indentação para definir blocos de código, o que já enfatiza a importância de indentar corretamente. O PEP 8 estabelece 4 espaços como indentação padrão para cada nível de bloco (loops, condições, definição de funções etc.). Tabs não são recomendados, muito menos misturar tabs e espaços – isso causa erros de sintaxe imediatamente em Python (`IndentationError`) e torna o código inconsistente entre editores. Portanto, configure seu editor para inserir espaços ao pressionar Tab.

Um fato curioso: uma análise do Stack Overflow descobriu que desenvolvedores(as) que usam espaços ganham em média salários maiores que os que usam tabs. Brincadeiras à parte, isso pode refletir que usar espaços (e portanto seguir convenções amplamente adotadas) é típico de desenvolvedore(as) experientes ou projetos mais profissionais. Independentemente da correlação com salários, manter a consistência na indentação é fundamental: 4 espaços por nível, sempre. Ferramentas como o `editorconfig` podem automatizar isso em projetos compartilhados,

garantindo que todos os editores de texto dos colaboradores usem o mesmo padrão de indentação.

Outra diretriz importante do PEP 8 é limitar o comprimento das linhas de código para melhorar a legibilidade. O guia original sugere até 79 caracteres por linha de código e 72 para linhas de comentários/docstrings. Linhas muito longas dificultam a leitura (especialmente em side-by-side diffs de controle de versão) e podem exigir rolagem horizontal inconveniente. No entanto, vale notar que isso não é uma regra absolutamente rígida: muitos projetos modernos adotam limites um pouco maiores (90 ou 100 colunas) sem prejuízo – afinal, “estamos no século XXI e temos telas de alta resolução”, como brinca a documentação do Django. O próprio black utiliza 88 caracteres como limite padrão, argumentando ser um equilíbrio entre os 79 clássicos e 99 que alguns preferem.

A recomendação é: quebre linhas longas para se adequarem a ~80 caracteres, mas não sacrifique a legibilidade só para cumprir um número – por exemplo, não deixe uma chamada de função com muitos argumentos ficar visualmente confusa ao quebrar linhas. Use bom senso (e as configurações do projeto, se houver, já que alguns adotam 100 ou 120 colunas como padrão). Quando precisar quebrar uma expressão em várias linhas, prefira usar parênteses para abranger as quebras em vez da contrabarra “\”. Por exemplo:

```
# Quebra de linha recomendada usando parênteses:
resultado = (variavel_a + variavel_b + variavel_c +
             variavel_d + variavel_e)
# (Acima, os operadores ficam no final de cada linha e a próxima linha é
# indentada)

# Evite:
resultado = variavel_a + variavel_b + variavel_c + \
variavel_d + variavel_e # Quebra com '\' e indentação inconsistente
```

Código-fonte 3 – Quebra de expressão em várias linhas

Fonte: Elaborado pelo autor (2025)

No primeiro caso, o Python entende a continuação dentro dos parênteses e não precisamos da barra invertida explícita; além disso, a segunda linha foi indentada para alinhar com a abertura do parênteses, seguindo uma estética que o PEP 8 incentiva. Já o segundo caso, embora funcione, fica menos claro (a barra no final indica continuação, mas a indentação não apresenta alinhamento visual). O PEP 8 sugere

sempre quebrar antes de um operador binário (como o +), e não depois, para que seja óbvio ao leitor que a linha seguinte está ligada à anterior. Similarmente, a chave ou parêntese de fechamento deve normalmente ficar sozinho na sua própria linha se a expressão for multilinha, facilitando o emparelhamento visual. São detalhes sutis, mas que contribuem para uma formatação agradável e padronizada.

Espaços em branco merecem menção especial. Um código “espremido”, sem espaços, dificulta a leitura tanto quanto um texto sem pontuação. Por isso, o PEP 8 recomenda: coloque espaços ao redor de operadores binários (atribuição =, aritméticos + - * /, comparadores == != < etc.) e após vírgulas em listas de parâmetros ou elementos. Exemplo: escreva `x = a + b` em vez de `x=a+b`; escreva `func(a, b, c)` em vez de `func(a,b,c)`. Por outro lado, não coloque espaços desnecessários em torno de parênteses, colchetes ou antes de vírgulas. Ou seja, `func(1, 2)` é correto, mas `func( 1, 2 )` (com espaços logo após (ou antes de)) é desencorajado. Essas regras tornam a separação de tokens mais uniforme – qualquer desenvolvedor(a) Python acostuma os olhos a ver `variavel = valor` e identificar imediatamente a atribuição, ou a ver `f(x, y)` e entender os argumentos. O uso disciplinado de espaços atua como pontuação visual do código.

Além disso, seguir consistentemente convenções de espaçamento evita diferenças supérfluas em diffs de controle de versão. Um espaço a mais ou a menos pode não mudar o funcionamento do código, mas gera diffs e conflitos desnecessários em merge. Equipes grandes às vezes sofrem com “whitespace changes” poluindo o histórico – o Django, por exemplo, orienta remover espaços em branco ao final de linhas porque além de não terem propósito, atrapalham patches e merges. Muitos editores e linters já alertam ou limpam whitespace sobrando automaticamente.

Quebra de blocos e organização: o PEP 8 também especifica quantas linhas em branco usar para separar certas estruturas. Deve-se usar duas linhas em branco entre definições de funções ou classes de nível top (ou seja, no módulo) e uma linha em branco entre métodos dentro de uma classe. Também é indicado inserir linhas em branco para separar “pedaços” lógicos dentro de uma função, quando pertinente, para melhorar a legibilidade.

No nosso exemplo de código não formatado (Código-fonte 1), a ausência de linhas em branco antes da definição da função violava essa regra (geraria um erro E302 no `pycodestyle`). Após corrigir, colocamos duas quebras antes de `def`

soma_valores, deixando claro visualmente onde começa a função. Essa separação facilita “respirar” o código, do mesmo modo que um texto bem estruturado tem parágrafos separados.

Comentários e Docstrings: escrever código legível não elimina a necessidade de comentários, mas diminui bastante a quantidade deles, já que o código passa a ser autoexplicativo em boa parte. Ainda assim, comentários são peças importantes para explicar porquês ou fornecer contexto adicional. O PEP 8 oferece algumas diretrizes para comentários: devem ser claros, concisos e úteis, preferencialmente em frases completas. Devem ser escritos em português (ou língua do projeto) correto e preferivelmente separados do código por uma linha acima (não ao lado do código, exceto comentários muito curtos). Comentários inline (no fim da linha) devem ser usados com moderação e, se usados, separados por pelo menos dois espaços do código. Evite comentários óbvios que apenas repetem o que o código já diz.

Por exemplo, não há por que comentar `x = x + 1 # Soma 1 ao x` – isso qualquer um entende. Já um comentário explicando decisões ou consequências é valioso, por exemplo: `x = x + 1 # incrementa para evitar dividir por zero na próxima etapa`. Além disso, mantenha comentários atualizados: um comentário desatualizado pode ser pior que nenhum (gera falsa compreensão). Em suma, comente o que for necessário para compreensão, nem mais nem menos.

Já as docstrings são cadeias de documentação dentro do código, usadas principalmente para documentar módulos, classes, métodos e funções públicas. Elas devem estar logo após a definição (entre aspas triplas `"""`) e idealmente explicar o que a função/classe faz, quais são seus parâmetros e retorno e se há efeitos colaterais. No hands on, incluímos uma docstring breve em `soma_valores` ilustrando esse conceito. Docstrings seguem convenções próprias (PEP 257), mas de forma geral: primeira linha resumida, no imperativo (“Calcula a soma...” em vez de “Esta função calcula...”), seguida de detalhes adicionais se necessário. Para projetos de ciência de dados, boas docstrings em funções de carregamento ou processamento de dados, por exemplo, podem indicar formatos esperados, unidades etc., ajudando outros a usarem corretamente seu código.

Recomendações de programação (boas práticas): o PEP 8 também traz algumas sugestões de boas práticas que extrapolam formatação, entrando um pouco em estilo de código no sentido de escolhas de implementação. Por exemplo, ele

recomenda não compararmos valores booleanos a True/False diretamente (usar `if cond`: ao invés de `if cond == True`;) e preferir `is not None` a `!= None`. Também orienta evitar imports do tipo `from module import *`, pois poluem o namespace e dificultam rastreabilidade (explícite sempre o que está importando). Aconselha dar preferência a generators ao invés de listas quando iterar sobre muitos elementos, por eficiência, e outras dicas que, embora não afetem a estética imediata, contribuem para um código mais Pythonico e robusto. Em essência, essas recomendações servem para manter a coerência de comportamento e a eficiência do código. Embora não entremos em cada detalhe aqui, é bom saber que o PEP 8 abrange também aspectos de lógica e não apenas aparência.

Por quê tudo isso importa? Podemos nos perguntar: seguir todas essas regras realmente vale a pena? A resposta da indústria e da pesquisa é um sonoro “sim”. Um estilo de código consistente traz inúmeros benefícios de engenharia de software:

- **Legibilidade e Manutenibilidade:** um código padronizado é como uma biblioteca bem organizada. Quando você ou um colega retornam a um script depois de meses, a consistência torna mais fácil entender o que está acontecendo. Estudos mostram que tempo de manutenção domina o ciclo de vida do software – ou seja, passamos mais horas lendo e ajustando código existente do que escrevendo do zero. Portanto, investir em legibilidade economiza tempo no longo prazo. Em contexto acadêmico ou de ciência de dados, pense em compartilhar um notebook ou script com colegas: se todos seguem convenções, o entendimento flui sem necessidade de reunião extra para “decifrar” o estilo pessoal de cada um.
- **Colaboração em Equipe:** em projetos com múltiplos desenvolvedores(as), seguir um padrão comum é fundamental. Caso contrário, cada um “toca uma música diferente” e o resultado é cacofonia no código. Convenções atuam como acordes harmonizando esse trabalho coletivo. Com um estilo consistente, code reviews se concentram em lógica e qualidade, não em detalhes de formatação. Isso reduz atritos: dificilmente haverá comentários do tipo “faltou espaço aqui, faltou ali” se um linter já garante isso automaticamente.

Aliás, uma estatística interessante: equipes que adotaram revisões sistemáticas de código, combinadas com ferramentas de análise estática, reduziram defeitos em mais de 35%. E um relato de caso real mostrou que uma startup que enforce consistência em nomes e tamanho de funções via hooks de pre-commit teve novos desenvolvedores contribuindo plenamente em uma semana em vez de um mês – a curva de aprendizado caiu drasticamente graças à padronização. A consistência elimina a necessidade de “traduzir mentalmente” estilos diferentes ao ler partes escritas por pessoas diversas.

- **Redução de Erros:** código inconsistente é terreno fértil para bugs. Imagine duas funções fazendo coisas semelhantes de jeitos totalmente distintos – é mais fácil não perceber uma condição faltando em uma delas. Por outro lado, abordagens padronizadas realçam anomalias. Se todo código segue um padrão, qualquer trecho fora do comum acende um alerta. Além disso, certas convenções evitam erros clássicos: por exemplo, não usar `l` como nome de variável evita confundir com `1` em fontes parecidas, prevenindo interpretações erradas. Indentação correta previne bugs lógicos (um bloco mal indentado pode executar fora do `if` esperado, por exemplo). Até mesmo merge conflicts ficam menos frequentes quando todos editam código formatado igual – ninguém vai reformatar tudo à sua maneira e entrar em conflito com mudanças de outros. Em suma, padrão diminui surpresa e menos surpresas significam menos bugs.
- **Qualidade e Profissionalismo:** seguir um estilo consagrado como PEP 8 demonstra profissionalismo e preocupação com qualidade. Em contextos de código aberto, os maintainers usualmente exigem conformidade com PEP 8 para aceitar contribuições; em empresas, código fora do padrão dificilmente passa da revisão. Assim, dominar essas convenções é praticamente obrigatório para quem quer trabalhar com Python no mercado. Muitos recrutadores e entrevistadores olham com bons olhos candidatos que escrevem código limpo e padronizado – é um sinal de maturidade técnica. Por outro lado, um repositório cheio de gambiarras visuais tende a ser malvisto e até descartado em avaliações.

- **Facilidade de Uso de Ferramentas:** código consistente também potencializa o uso de ferramentas automáticas de análise e refatoração. Parsers e analisadores estáticos conseguem fazer um trabalho melhor quando o código segue certas expectativas de formatação. Além disso, quem já abriu um PR no Github talvez tenha visto bots como pep8speaks ou Stickler CI comentando problemas de estilo – integrar essas ferramentas é fácil quando se adota PEP 8, pois elas já sabem o que checar. Projetos de data science, por exemplo, muitas vezes envolvem notebooks Jupyter, onde é tentador afrouxar estilo. Mas hoje existem linters integrados no Jupyter também e manter padrão nos notebooks facilita eventualmente converter aquele experimento em um script ou módulo reutilizável.

Ferramentas de Linting e Formatação Automática: felizmente, não precisamos verificar manualmente cada regra do PEP 8 em todo código que escrevemos. Seria impraticável (além de entediante) ficar contando espaços ou caçando linhas longas toda hora. É aqui que entram as ferramentas de linting/formatação, que automatizam a checagem e até a correção do estilo de código.

- **Flake8:** é uma ferramenta de lint leve muito popular na comunidade Python. Essencialmente, ela combina três verificadores em um só: o PyFlakes (que procura erros simples no código, como variáveis não usadas), o pycodestyle (anteriormente chamado pep8, que verifica estritamente conformidade ao PEP 8 em termos de formatação) e o McCabe (que calcula a complexidade ciclomática para detectar funções muito complexas). Assim, ao rodar flake8 em um arquivo, ele varre o código sinalizando tanto problemas de estilo (E302 para linhas em branco faltando, E231 para espaço faltante etc.) quanto potenciais bugs simples ou code smells (como uma variável definida e não usada, ou complexidade acima do limite).

O flake8 é altamente configurável via plugins e permite ignorar regras específicas conforme a necessidade. Por padrão, ele já ajuda a cumprir boa parte do PEP 8. No hands on, mencionamos alguns erros prefixados por “E” (de Error) que um pycodestyle detectou; o flake8 apresenta exatamente essas siglas nas mensagens. Uma execução típica do flake8 no nosso código inicial retornaria, por exemplo: E302 expected 2 blank lines, found 0

e E231 missing whitespace after ',', entre outras, orientando as correções. Apesar do nome, o flake8 não conserta nada sozinho – ele apenas aponta onde o código “flakeia” nas normas.

- **Pylint:** é um linter mais abrangente e estrito. Enquanto o flake8 está focado principalmente em estilo e alguns erros simples, o pylint realiza uma análise estática completa: verifica convenções de nome (e.g., vai reclamar de função com letra maiúscula), formatação, erros (referências a variáveis inexistentes etc.), dá alertas de code smells (por exemplo, muitos branches aninhados, complexidade, duplicação de código) e até sugestões de melhorias. O pylint atribui um “score” ao código (de 0 a 10 geralmente), que começa em 10 e vai decrescendo conforme encontra problemas.

Ele pode, por exemplo, dar nota 8.5/10 para um módulo e listar dezenas de mensagens categorizadas (C para convention, W para warning, E para error, R para refactor, F para fatal). Entre elas estariam coisas como “C0103: variável X não segue o nome em minúsculas” ou “W0612: variável foo atribuída mas não usada”. O pylint é conhecido por ser verborrágico – inicialmente, quase nenhum arquivo atinge nota 10 sem algum ajuste ou filter. A grande vantagem é que ele ajuda a elevar a qualidade geral porque pega desde detalhes de estilo até possíveis bugs lógicos. Por exemplo, ele detecta se você importou um módulo e não usou (evitando desperdício de memória), ou se há trecho idêntico de código copiado em locais diferentes, sugerindo refatoração.

Por baixo dos panos, o pylint constrói um AST (Abstract Syntax Tree) do seu código e faz análises profundas, até entre múltiplos arquivos (diferente do pyflakes do flake8, que olha cada arquivo isoladamente). Ele é altamente configurável – você pode ajustar o .pylintrc para ignorar certas regras ou alterar padrões (por exemplo, dizer que nomes com 1 letra são permitidos em determinados casos etc.). Muitos projetos legados adotam pylint gradualmente: primeiro ajustam o código ao básico do flake8, depois introduzem pylint para ir limpando problemas mais sutis. Por ser mais lento (analisa muito mais coisa que o flake8), algumas equipes rodam pylint só no CI ou periodicamente, enquanto o flake8 roda em cada commit. Ainda assim, é uma ferramenta poderosa. Conforme um artigo recente, o pylint

continua sendo uma escolha de referência para análise minuciosa, especialmente em projetos grandes onde seus relatórios detalhados agregam muito valor na manutenção da qualidade.

- **Black:** diferente de flake8 e pylint, que apenas alertam sobre problemas, o Black é um formatador automático (code formatter). Seu lema é ser um formatador “intransigente” – ele vai reescrever todo o seu código em um estilo padrão, automaticamente, sem precisar de configuração. Ao rodar `black meu_codigo.py`, ele reformata o arquivo seguindo as convenções: adiciona/remove espaços conforme PEP 8, quebra linhas longas, arruma indentação etc., tudo de forma determinística. Importante: o Black não altera a lógica do código, só a formatação. Ele inclusive ignora partes que não conseguir formatar sem mudar a semântica.

Por exemplo, se você tiver uma string multilinha mal indentada, ele não arrisca alterar o conteúdo dela. A proposta do Black é eliminar debates sobre estilo em código – você aceita o estilo que ele impõe (ligeiramente adaptado do PEP 8, como a linha de 88 caracteres) e pronto. Isso traz duas vantagens enormes: (1) desenvolvedores(as) não perdem tempo discutindo se algo deve ter 1 ou 2 espaços ou onde quebrar a linha – deixam o Black decidir; (2) o diffs de código ficam mais limpos, pois ao aderir ao Black, toda a base de código é formatada consistentemente. Nas próximas mudanças, diffs mostram apenas alterações lógicas, não mudanças cosméticas.

Muitas equipes configuram o Black para rodar automaticamente ao salvar o arquivo no editor ou no pre-commit. Assim, o código já é salvo “bonito”. No dev.to citado, o Black é descrito como uma ferramenta que “aplica um estilo consistente e predefinido ao código sem exigir configuração adicional” e pode integrar-se ao fluxo de trabalho formatando o código a cada salvamento ou envio ao versionamento. Uma observação: o Black não verifica nomes ou lógica – ele não vai renomear `SomaValores` para `soma_valores`, nem avisar se você deixou uma variável sem uso.

Portanto, é comum usar Black em conjunto com flake8/pylint. O Black corrige a formatação grosseira e então os linters ficam encarregados de aspectos que o Black não cobre (como naming ou possíveis bugs). A

sinergia dessas ferramentas eleva muito a aderência ao padrão: de um lado, o Black garante 100% conformidade em indentação, espaços, quebras de linha; de outro, flake8/pylint garantem as demais regras e boas práticas.

- **Outras ferramentas úteis:** além de flake8, pylint e black, existem mais utilitários no ecossistema. O isort, por exemplo, lida especificamente com organização de imports – ele ordena automaticamente os imports em ordem alfabética e separa em grupos (built-in, terceiros, locais) conforme convenção, complementando o PEP 8 que recomenda uma ordem lógica de imports. Em projetos grandes, manter imports organizados manualmente é trabalhoso; o isort resolve em segundos. Outra ferramenta é o autopep8, que formata o código automaticamente para aderir ao pep8 (foi criada antes do Black; funciona bem, mas hoje muitos preferem o Black pela eficácia consistente).

Há também o pep8speaks (bot de GitHub que comenta violações pep8 em PRs), o pre-commit (um framework para rodar linters/formatters antes de commits – muito usado para automatizar Black, Flake8 etc. em repositórios) e até linters mais novos e ultrarrápidos como o Ruff. O Ruff, aliás, é uma tendência recente digna de nota: escrito em Rust, ele roda cerca de 100x mais rápido que linters tradicionais e implementa boa parte das regras do flake8, pylint e até do Black em um só pacote. Com o Ruff, grandes codebases (de centenas de milhares de linhas) podem ser lintadas em menos de um segundo, o que é impressionante. Projetos como o Pylint já usam o Ruff internamente para acelerar certas verificações. É possível que nos próximos anos ele se torne o padrão, mas por enquanto flake8/pylint ainda reinam pela maturidade e compatibilidade ampla.

Para comparar essas ferramentas de forma direta, veja a tabela a seguir:

FERRAMENTA	TIPO	FUNÇÃO PRINCIPAL	EXEMPLO DE USO
Flake8	Lint de estilo	Verifica aderência ao PEP8 e erros simples (syntaxe etc.) combinando PyFlakes, pycodestyle e McCabe. Leve e extensível via plugins (ex.: flake8-bugbear). Não formata, apenas acusa problemas.	flake8 script.py – exibe linhas e códigos de erro (E,W,F) indicando violações de estilo e possíveis bugs.
Pylint	Lint estático	Análise estática extensa: checka estilo PEP8, erros, "code smells" e padrões de design. Altamente configurável (pylintrc) e fornece nota de qualidade do código. Útil para garantir qualidade em projetos grandes, embora mais lento.	pylint meu_modulo.py – gera um relatório com mensagens de convenção (C), aviso (W), erro (E), refatoração (R) etc. e uma nota final (ex.: 8.5/10).
Black	Formatador	Formata automaticamente o código conforme um estilo padronizado (base PEP8, opinativo). Sem necessidade de configuração; muito rápido. Garante consistência imediata na formatação (espaços, quebras, aspas).	black projeto/ – reescreve in place todos os arquivos .py do diretório projeto no estilo padronizado (88 col/4 espaços etc.).

isort	Organizador imports	Ordena e agrupa declarações import automaticamente, conforme convenções (alfabética e por seções). Mantém os imports limpos e padronizados – frequentemente usado junto com Black.	isort meu_modulo.py – reordena os imports no arquivo, separando em blocos de stdlib, third-party e locale, conforme configuração.
-------	---------------------	--	---

Tabela 2 – Comparativo de ferramentas de linting/formatação em Python e seus usos
 Fonte: Elaborado pelo autor (2025) com informações dos sites oficiais e literatura técnica

Integrar essas ferramentas no fluxo de desenvolvimento é uma das melhores coisas que você pode fazer pela saúde do seu código. Por exemplo, você pode configurar um hook de pre-commit que rode o Black e o flake8 toda vez que um commit for efetuado – se o flake8 encontrar um erro grave ou o Black fizer alterações, o commit é abortado, forçando você a corrigir antes de prosseguir. Isso automatiza a disciplina: com o tempo, seu código já nasce no padrão porque você se habitua ao corretor “ortográfico” dele. Editores modernos e IDEs oferecem integração nativa: o VSCode tem extensões para flake8/pylint que sublinham problemas em tempo real e pode formatar com Black ao salvar; o PyCharm destaca violações de PEP8 como inspeções de código; até notebooks Jupyter, via extensões, conseguem rodar black/flake8 no código das células. Ou seja, o(a) desenvolvedor(a) nem precisa lembrar de rodar a ferramenta manualmente – o ambiente de desenvolvimento bem configurado faz isso de fundo, quase como um compilador de qualidade.

Do ponto de vista de ciência de dados, manter um estilo coeso ajuda na reprodutibilidade e transferência de projetos. Imagine um pesquisador de dados que prototipou um modelo fantástico, mas com um código bagunçado, difícil de entender e manter. Ao passar esse código para a equipe de engenharia implementar em produção, haverá dificuldade e retrabalho. Se esse protótipo já estivesse escrito com boas práticas, seguindo PEP8, usando nomes claros etc., a transição ciência -> engenharia seria bem mais suave. Cada vez mais fala-se em “DataOps” e “MLOps”, integrando data science ao desenvolvimento de software – e isso inclui adotar padrões de código no trabalho exploratório também. Portanto, até em notebooks experimentais

vale a pena usar um linter e formatador (existem versões como o nbblack para Jupyter). Pode parecer excesso de zelo durante uma análise ad-hoc, mas quando você revisitar aquele notebook meses depois ou precisar compartilhá-lo, vai agradecer por ele estar organizado.

Flexibilidade e bom senso são pontos fundamentais: apesar de todas essas regras, o PEP 8 não é uma lei inflexível. Na verdade, o próprio texto do PEP 8 diz: “Em caso de conflito, mantenha a consistência do código existente”. Ou seja, se você estiver contribuindo em um projeto que deliberadamente adota um estilo ligeiramente diferente em algum aspecto, siga o padrão local em vez de aplicar cegamente o PEP 8. Por exemplo, o Django, como vimos, opta por linhas mais longas quando isso ajuda a legibilidade do código específico deles. Outros podem adotar 2 espaços de indentação em um contexto especial (não comum em Python, mas hipoteticamente).

O objetivo final é tornar o código uniforme dentro do projeto. Então, use o PEP 8 como guia base – ele cobre 95% dos casos – mas aplique julgamento de engenheiro(a) para saber quando uma exceção é aceitável. Se uma regra do estilo tornar o código realmente pior em determinada situação, documente a exceção e siga em frente. Por exemplo, talvez uma tabela ASCII gigante embutida no código não deva ser quebrada em 79 colunas. Ou um nome de variável muito consagrado internacionalmente (uma fórmula matemática, tipo *Re* para número de Reynolds) pode ser mantido assim, apesar de fugir do padrão, por clareza no domínio científico. O PEP 8 inclusive menciona: “às vezes a convenção de código do contexto é mais importante do que seguir estritamente estas regras”.

No entanto, enquanto iniciante ou em projetos novos, é recomendável aderir estritamente ao PEP 8 até desenvolver bom senso suficiente para saber quando e como excepcionar. “Conheça as regras antes de quebrá-las”. E se quebrá-las, que seja por razões sólidas e raramente. Na dúvida, discuta com o time – muitas equipes documentam em um CONTRIBUTING.md ou similar quais regras do PEP 8 elas não seguem ao pé da letra.

Vale comentar que segurança de software e performance podem ter ligação indireta com estilo também. Código confuso pode esconder vulnerabilidades ou tornar mais difícil aplicar patches. Já um código claro e padronizado facilita auditorias e otimizações – por exemplo, detectar um trecho ineficiente é mais fácil se o estilo está consistente em todo lugar (você consegue achar rapidamente todas as ocorrências

daquele padrão). Não por acaso, muitas listas de Secure Coding incluem “siga um guia de estilo” como prática para evitar descuidos.

Para praticar mais: experimente rodar o flake8 em um projeto seu e veja os resultados. Em seguida, use o black nele e observe as diferenças no código formatado. Essa experiência hands on reforçará os conceitos e mostrará na prática como as ferramentas agem. Não deixe também de revisitar a videoaula e o material de apoio sempre que surgir dúvida – padrões se solidificam com repetição e prática deliberada. Futuramente, você verá outros aspectos de qualidade de código e como eles se conectam com essas convenções de estilo.

MERCADO, CASES E TENDÊNCIAS

No cenário profissional atual, é praticamente unânime a importância de manter padrões de código. A filosofia “cada um codifica do seu jeito” ficou para trás – hoje, times de alta performance investem em code style guides, linters no pipeline de CI e formataadores automáticos para garantir qualidade e velocidade. Vamos ver alguns exemplos e tendências ligados a convenções de estilo em Python:

Adoção Universal do PEP 8: o guia de estilo PEP 8 tornou-se referência obrigatória na comunidade Python. Grandes projetos open-source exigem conformidade com ele. Por exemplo, o framework Django, em suas instruções de contribuição, enfatiza: “siga o PEP 8” e recomenda usar a ferramenta pep8 (pycodestyle) para verificar antes de enviar código. Muitos outros projetos (Pandas, SciPy, Flask etc.) têm verificações automáticas de estilo nos seus testes de integração contínua – um merge request que introduz espaços a mais ou nomes fora do padrão provavelmente falhará nos checks. Isso disciplina contribuidores ao redor do mundo e mantém esses grandes codebases coesos, mesmo com centenas de pessoas editando. Empresas também convergiram para o PEP 8 como base; o Google, por exemplo, possui um guia interno de Python muito similar ao PEP 8 (com poucas diferenças específicas). Em suma, conhecer e aplicar o PEP 8 é esperado de qualquer desenvolvedor(a) Python profissional hoje.

Ferramentas Automáticas no Fluxo de Trabalho: uma forte tendência é integrar cada vez mais ferramentas como Black, flake8/pylint e isort diretamente nos editores e fluxos de CI/CD. O GitHub, por exemplo, facilita configurar GitHub Actions que rodam Black e linters em todos os pushes. Há também o popular pre-commit framework – bastam algumas linhas de config para que, antes de cada commit local, seu código seja formatado e checado automaticamente. Equipes que adotam isso relatam ganhos enormes de produtividade em revisões de código: um case reportou que, após adicionar o Black e flake8 no pre-commit, as revisões reduziram em ~40% os comentários triviais (pontos e vírgulas do estilo), permitindo focar no conteúdo.

Outro exemplo: a Microsoft adotou o Black em projetos como o VSCode Python extension, e desenvolvedores(as) notaram menos debates em PRs sobre formatação. Ou seja, tornou-se best practice delegar ao tooling aquilo que pode ser automatizado.

Também emergem ferramentas integradas – o já citado Ruff está ganhando popularidade rapidamente, pois consolida múltiplas funções: ele detecta estilo, alguns erros e formata parcialmente, tudo com desempenho impressionante. Lançado em 2022, em 2025 já é utilizado em grandes codebases (p. ex., o Pydantic, projeto de validação de dados em Python, migrou para o Ruff para acelerar linting). É provável que nos próximos anos vejamos uma convergência para ferramentas unificadas e extremamente rápidas, simplificando ainda mais manter padrões.

Cultura de Código Limpo: a influência de livros como “Clean Code” (Robert Martin, 2008) e movimentos de software craftsmanship reforçou a ideia de que código deve ser escrito para humanos lerem, não apenas para máquinas executarem. Isso ressoou fortemente no mercado. Hoje, muitas organizações oferecem treinamentos internos de estilos e melhores práticas e novas vagas frequentemente listam “escrever código limpo e manutenível” como requisito. Em Python isso invariavelmente significa escrever código Pythonico e seguir o PEP 8. Casos de sucesso abundam: um artigo da MoldStud Research (2025) relata diversos projetos onde introduzir revisão de código sistemática e ferramentas de análise reduziu defeitos em ~35% e o backlog de bugs em 40%.

Um ponto citado foi a padronização de convenções via linters já no início do ciclo de desenvolvimento – isso traz ganhos imediatos de qualidade. Na contramão, também ouvimos sobre casos de fracasso onde a falta de padrões cobrou seu preço: por exemplo, a empresa eBay, anos atrás, teve que refatorar uma grande base Python legada onde cada módulo tinha um estilo próprio – desenvolvedores(as) perdiam dias entendendo código alheio devido a inconsistências. Tiveram que instituir um mutirão de limpeza de código (introduzindo pep8 e pylint gradativamente) para domar a dívida técnica. Esse tipo de “déficit de estilo” é um tipo de technical debt pouco visível do qual as empresas estão cada vez mais conscientes.

Academia e Ciência de Dados se adaptando: tradicionalmente, ambientes acadêmicos ou de pesquisa tinham reputação de escrever código mais livre, focado só em funcionar. Porém, isso vem mudando: grupos de pesquisa percebem que código científico é mais útil quando outros conseguem ler e reutilizar. Há iniciativas de ensino de boas práticas em computação científica que incluem seguir guias de estilo. Por exemplo, alguns afirmam que pesquisadores que adotaram linters viram uma

redução de erros em scripts de análise e uma melhor replicabilidade dos experimentos (pois colegas entendiam e executavam o código com menos tropeços).

Em data science corporativo, esse alinhamento é ainda mais claro: empresas querem que os modelos e análises feitos por times de DS possam ser levados à produção e mantidos por engenheiros, o que requer aderência a padrões de código desde a fase exploratória. Não à toa, vemos conteúdo como “PEP8 for Data Scientists” sendo publicado, salientando que seguir PEP 8 em pipelines de ML facilita integração e colaboração no time. Em resumo, as fronteiras entre “código de pesquisa” e “código de produção” estão se esvaindo – todos convergem para a ideia de código claro, legível e padronizado.

Conteúdo e Comunidade: a comunidade Python produz constantemente material sobre boas práticas de estilo. O Real Python, por exemplo, além do tutorial completo, disponibilizou quizzes interativos e cheatsheets para fixar PEP 8. Conferências Python (PyCon etc.) frequentemente têm palestras sobre “Escrevendo código Python profissional” ou estudos de caso de adoção de Black/linters em grandes sistemas. No YouTube, existe uma série de vídeos populares mostrando “antes e depois” de formatar código ruim para bom (Mosh, Corey Schafer etc., lançaram conteúdos assim).

Até memes e humor refletem isso – piadas sobre “indentar com 3 espaços” ou “esquecer de rodar o Black” são comuns em fóruns, o que denota como a cultura internalizou esses hábitos. Esse engajamento comunitário ajuda os desenvolvedores e desenvolvedoras a aprenderem de forma leve. Um meme recente mostrava a reação de horror ao ver um colega submeter um código todo desalinhado e sem PEP8 – obviamente exagerado para rir, mas revelador de que hoje isso é visto como gaffe (uma gafe) no meio profissional.

Em termos de tendência futura, podemos esperar que ferramentas de linters se tornem ainda mais inteligentes, incorporando talvez análises com IA para sugerir refatorações melhores (já existem experimentos em IDEs com assistentes de código que não só formatam, mas reescrevem pedaços para ficarem mais claros). Contudo, a base dessas sugestões continuará sendo as convenções consagradas – dificilmente aparecerá algo que destoe radicalmente do PEP 8, pois ele se provou eficaz por mais de duas décadas. O PEP 8 em si pode evoluir para acomodar novas sintaxes (por exemplo, com a chegada de pattern matching no Python 3.10, talvez sejam

adicionadas recomendações de estilo específicas para esse recurso). Mas o espírito permanece: legibilidade e consistência acima de preferências individuais.

Para concluir esta seção, vale lembrar que adoção de padrões traz consigo melhorias mensuráveis: equipes ficam mais sincronizadas, o código causa “menos surpresa” (The Principle of Least Astonishment) e a manutenção se torna proativa em vez de reativa. Empresas como Microsoft, Google e Spotify, entre outras, já declararam abertamente que investem em qualidade interna de código, pois isso reduz custos a longo prazo. Assim, dominar e aplicar convenções de estilo não é apenas uma questão acadêmica; é seguir as pegadas de sucesso de quem constrói softwares robustos e colaborativos.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a fundo como as Naming Conventions e o Estilo de Código em Python impactam a qualidade de projetos. Aprendemos as principais convenções do PEP 8 – desde nomes de variáveis, funções, classes e constantes, até padrões de indentação, limite de linha, uso de espaços e organização de código – entendendo como cada guideline contribui para tornar o código mais legível, uniforme e manutenível.

Vimos, na prática, a transformação de um código “bagunçado” em um código bem estruturado, aplicando essas convenções e usando ferramentas automáticas. Identificamos problemas comuns de estilo e utilizamos linters (como flake8/pylint) para detectá-los e formatadores (como Black) para corrigi-los, passo a passo.

Aprofundamos o embasamento técnico por trás dessas boas práticas, ligando-as a princípios de engenharia de software, como colaboração em equipe, redução de erros e facilitação de revisões. Discutimos como um estilo consistente atua como documentação implícita do código e até como pode afetar positivamente o desempenho de desenvolvimento (evitando dívidas técnicas e retrabalho).

Conhecemos um leque de ferramentas que auxiliam na garantia de estilo: linters (flake8, pylint, Ruff) e formatadores (Black, autopep8, isort), comparando suas características e melhores usos. Entendemos a importância de integrá-las ao fluxo de trabalho para manter a qualidade de forma contínua e automatizada.

Por fim, analisamos casos do mundo real e tendências: organizações reais colhendo benefícios de adotar padrões de código (menos bugs, onboarding mais rápido, maior produtividade), a comunidade Python inteira convergindo em torno do PEP 8 como referência e a modernização do ecossistema com novas ferramentas rápidas e cultura de código limpo. Também vimos que até em ciência de dados, onde antes reinava certa informalidade, hoje a profissionalização exige seguir convenções para viabilizar projetos de longo prazo.

Em resumo, você deve agora estar convencido(a) de que escrever código Python não é só “fazer funcionar”. Importa como escrever. Ao aplicar consistentemente as convenções de nomenclatura e estilo certas, você não apenas deixa seu código bonito – você torna seu programa mais confiável, compreensível e

sustentável. Este é um aprendizado fundamental para elevar seu nível como desenvolvedor(a) Python profissional ou pesquisador(a) que codifica: o código é uma comunicação e seguir um padrão reconhecido é falar de forma clara com todos os que lerão seu trabalho.

EMENDAS

REFERÊNCIAS

CRUDU, A.; MOLDSTUD RESEARCH. **Real-Life Case Studies – Boosting Python Code Quality in Projects**. 2025. Disponível em: <https://moldstud.com/articles/p-real-life-case-studies-boosting-python-code-quality-in-projects> . Acesso em: 21 jan. 2026.

DJANGO SOFTWARE FOUNDATION. **Coding style (Django Documentation) – Section “Python style”**. 2017. Disponível em: <https://docs.djangoproject.com/en/dev/internals/contributing/writing-code/coding-style/> . Acesso em: 21 jan. 2026.

FINER, J. **How to Write Beautiful Python Code With PEP 8**. 2025. Disponível em: <https://realpython.com/python-pep8/> . Acesso em: 21 jan. 2026.

RAMOS, V. **O guia de estilos de programação PEP8 do Python**. 2023. Disponível em: <https://pythonacademy.com.br/blog/o-guia-de-estilos-pep8-do-python> . Acesso em: 21 jan. 2026.

ROBINSON, D. **Developers Who Use Spaces Make More Money Than Those Who Use Tabs**. 2017. Disponível em: <https://stackoverflow.blog/2017/06/15/developers-use-spaces-make-money-use-tabs/> . Acesso em: 21 jan. 2026.

SOARES, C. H. G. **Domine a escrita de códigos Python com PEP8 e suas bibliotecas**. 2024. Disponível em: <https://dev.to/carlos-cgs/domine-a-escrita-de-codigos-python-com-pep8-e-suas-bibliotecas-1mp0> . Acesso em: 21 jan. 2026.

TRUNK. **Comparing Ruff, Flake8, and Pylint – Linting Speed**. 2024. Disponível em: <https://trunk.io/learn/comparing-ruff-flake8-and-pylint-linting-speed> . Acesso em: 21 jan. 2026.

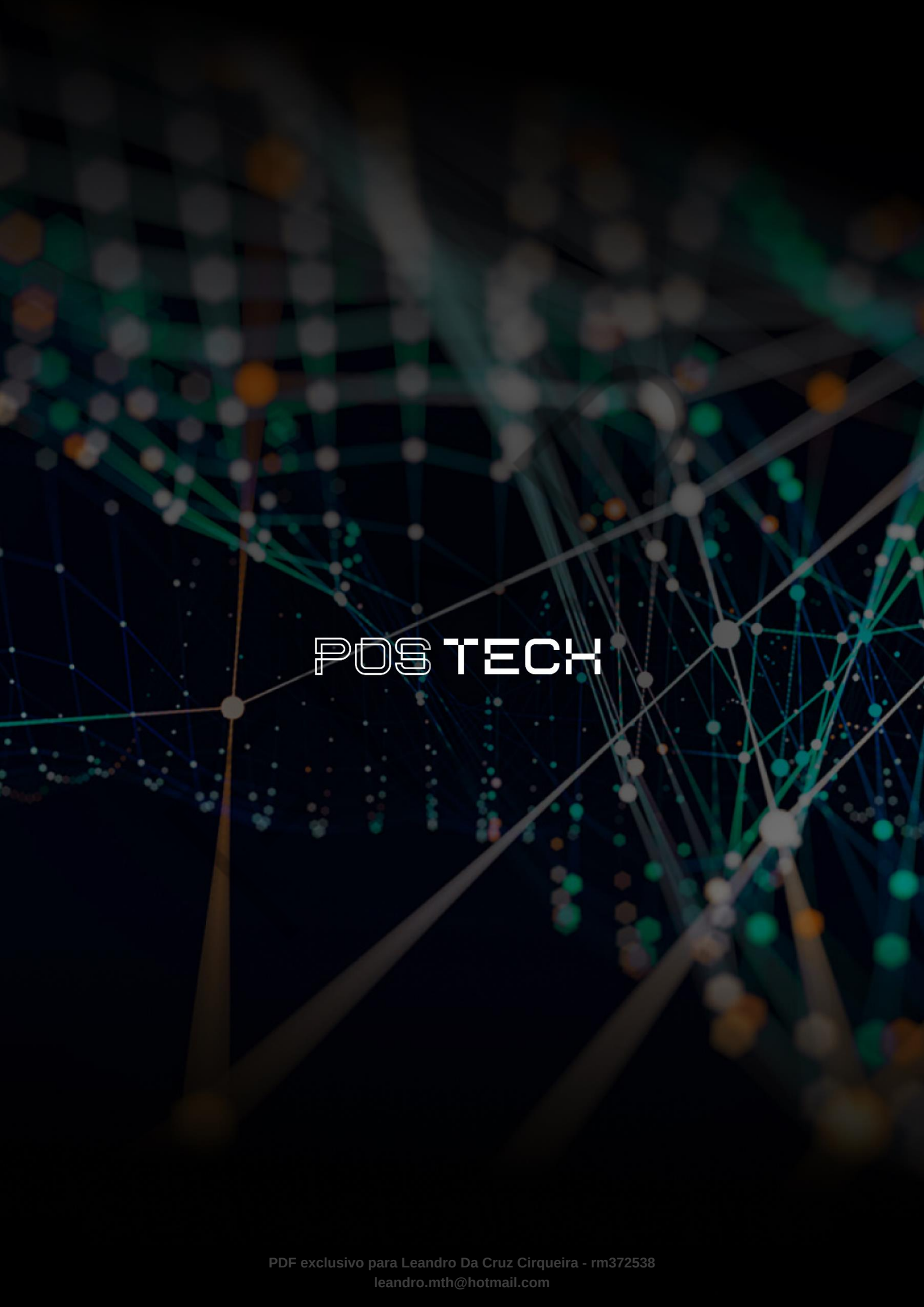
VAN ROSSUM, G.; WARSAW, B.; COGHLAN, A. **PEP 8 – Style Guide for Python Code**. 2001. Disponível em: <https://peps.python.org/pep-0008/> . Acesso em: 21 jan. 2026.

ZHIRNOV, M. **The Importance of Code Consistency in Team Projects**. 2024. Disponível em: <https://hemaks.org/posts/the-importance-of-code-consistency-in-team-projects/> . Acesso em: 21 jan. 2026.

PALAVRAS-CHAVE

Estilo de código. Legibilidade e manutenibilidade. Linting e formatação automática.

EMENDAS



POSTECH